

CCF直播

LMCC 认证 · Python 精讲

看懂大模型代码——
一堂课打通 LMCC 编程题的 Python 语法关卡

讲师：猿编程 李超 @bigbigli

今日直播议程

4 大模块

00

开场热身

为什么 LMCC 也要考 Python

01

语法快速体检

变量·条件·循环·函数，扫一遍地基

02

AI 代码里的高频写法 + 随堂小测

推导式 / lambda / *args/**kwargs / with / class

03

吃透 LMCC 真题代码 + 随堂小测

RAG 填空题 + LoRA 配置题 + 唐诗检索代码

04

手搓一个迷你 RAG

串起本课所有语法点，跑一遍完整流程

05

总结

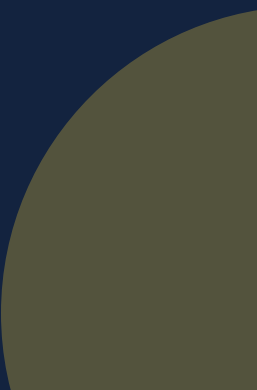
一张知识地图



00

开场热身

重新认识 Python?



LMCC 考的不是「刷题 Python」，是「读代码 Python」

以前学 **Python**: 写一个函数、判个奇偶、排个序，逻辑比较单一。

LMCC 里的 Python: 题目直接甩给你一段调用大模型 API 的代码，让你读懂、填空、改参数。

关键变化: 不需要你从零手写复杂算法，而是要求你认识"AI 圈子里"的常用写法——列表推导式、`*args/**kwargs`、`with` 语句、类的写法。

今天的目标: 把这些「面生」的写法扫一遍，再拿真题代码练手，做到看到类似代码不再懵。

一段真实的 LMCC 代码，你能看懂多少？

- 下面是 LMCC 第二轮真题中，检索古诗词的核心函数（稍作简化）：



小小预告：这段代码里藏了 5 个知识点

```
def retrieve_relevant_poems(problem, kb, top_k=3):  
    clean = re.sub(pattern, '', problem)  
    trigrams = {clean[i:i+3] for i in range(len(clean)-2)}  
  
    candidates = []  
    for poem in kb:  
        score = sum(overlap(...) for p in poem['paragraphs'])  
        candidates.append({'score': score, 'poem': poem})  
  
    candidates.sort(key=lambda x: x['score'], reverse=True)  
    return candidates[:top_k]
```

① 默认参数

`top_k=3`

② 集合推导式

`{ ... for ... in ... }`

③ 生成器表达式

`sum(... for ...)`

④ 字典 + 列表

组合数据结构

⑤ **lambda** 排序键

`key=lambda x: ...`

▸ 这 5 个点，我们接下来逐个拆解!

01

语法快速体检

把生疏的 Python 语法快速捡起来

Python 的几个「脾气」，先对齐一下

缩进即语法： Python 不用 {} 包代码块，靠缩进（一般4个空格）表示"谁属于谁"，缩进错了程序直接报错。

变量不用先声明类型： 写 `x = 10` 就直接能用，`x` 的类型是动态的，之后甚至可以把 `x` 变成字符串。

一切都是对象： 数字、字符串、函数、类，本质上都是"对象"，都能赋值给变量、当参数传递。

代码要「读得顺」： Python 社区很看重代码可读性，很多写法（推导式、`with`语句）就是为了让代码更短更清楚。

变量与基本数据类型

```
name = "Qwen2.5"      # 字符串 str
version = 2.5          # 浮点数 float
top_k = 3              # 整数 int
is_ready = True        # 布尔值 bool

tags = ["RAG", "LoRA"] # 列表 list, 可修改
config = {"lr": 1e-4, "epoch": 3} # 字典 dict, 键值对
seen = {"RAG", "SFT"}  # 集合 set, 自动去重
point = (1, 2)         # 元组 tuple, 不可修改
```

str / int / float / bool

最基础的四种类型

list

方括号 [], 能增删改, 最常用

dict

花括号 {}, 按「键」取值, 比如 config['lr']

set / tuple

集合自动去重; 元组一旦定义不能再改

条件与循环：靠缩进划分代码块

```
score = 0.82
```

```
if score >= 0.9:  
    label = "强相关"
```

```
elif score >= 0.6:  
    label = "相关"
```

```
else:  
    label = "不相关"
```

```
for i, poem in enumerate(kb):  
    print(i, poem["title"])
```

```
while len(candidates) < top_k:  
    candidates.append(fallback())
```

if / elif / else

冒号：后换行，下一行必须缩进

for i, x in enumerate(...)

同时拿到下标 *i* 和元素 *x*

while 条件:

条件为真就一直循环

- 没有 {}, 缩进就是唯一的分块方式

函数定义：一切从 def 开始

```
def build_prompt(question, context, top_k=3, verbose=False):  
    """根据检索结果拼一个提示词"""  
    parts = [f"问题: {question}"]  
    for i, c in enumerate(context[:top_k]):  
        parts.append(f"参考{i+1}: {c}")  
    prompt = "\n".join(parts)  
    if verbose:  
        print(prompt)  
    return prompt
```

def 函数名(参数):

定义函数的固定格式

top_k=3, verbose=False

默认参数：调用时可以不传，用默认值

f"问题: {question}"

f-string：在字符串里直接嵌变量，很常用

return

把结果送回调用的地方，没有return默认返回None

02

AI 代码里的高频写法

看懂大模型代码必须认识的 6 个写法

列表推导式：把 for 循环「压」成一行

传统写法

```
scores = []  
for item in candidates:  
    scores.append(item["score"])
```

列表推导式写法

```
scores = [item["score"] for item in candidates]
```

带条件筛选

```
good = [c for c in candidates if c["score"] > 0.5]
```

[表达式 for 元素 in 序列]

等价于「造一个新列表」

[表达式 for 元素 in 序列 if 条件]

多加一层筛选

- 读法：从右往左读——先看for再看表达式
- 本质：跟普通for循环做的事完全一样，只是更短

字典推导式 & 集合推导式

```
# 字典推导式: { key: value for ... }  
id2score = {c["id"]: c["score"] for c in candidates}  
  
# 集合推导式: { 表达式 for ... }, 自动去重  
trigrams = {text[i:i+3] for i in range(len(text)-2)}  
  
# 生成器表达式: ( 表达式 for ... ), 不会先造出完整列表  
total = sum(c["score"] for c in candidates)
```

{}包起来 + 冒号 → 字典推导式

{}包起来, 没有冒号 → 集合推导式

()包起来 → 生成器表达式
更省内存, 适合只用一次的场景

- 外形很像, 靠「括号类型」和「有没有冒号」来区分

lambda: 一句话就能定义的「临时函数」



```
# 完整定义一个函数
def get_score(item):
    return item["score"]

candidates.sort(key=get_score, reverse=True)

# 用 lambda 直接写在调用的地方, 不用单独起名字
candidates.sort(key=lambda item: item["score"], reverse=True)
top1 = max(candidates, key=lambda x: x["score"])
```

lambda 参数: 返回值

定义一个「只用一次」的小函数

最常见场景

sort / sorted / max / min 的 key 参数

- ▶ 本质: 跟def定义的函数是一回事, 只是不用起名字
- ▶ reverse=True 表示「从大到小」排序

*args / **kwargs: 不定数量的参数

```
def call_model(prompt, *args, temperature=0.7,
               **kwargs):
    print("位置参数: ", args)
    print("关键字参数: ", kwargs)
    print("温度: ", temperature)

call_model("你好", "额外1", "额外2", top_p=0.9,
          max_tokens=512)
# args = ("额外1", "额外2")
# kwargs = {"top_p": 0.9, "max_tokens": 512}
```

*args

打包成「元组」，接收多个没写名字的参数

**kwargs

打包成「字典」，接收多个「名字=值」的参数

- ▶ 常见场景：模型API的调用函数，参数种类多、可选项多
- ▶ 调用时也能反过来「解包」：func(*my_list, **my_dict)

with 语句：自动帮你「收尾」

```
# 不用 with: 容易忘记关闭文件
f = open("data.json")
content = f.read()
f.close()    # 一旦忘记写, 或者中间报错, 文件就没关

# 用 with: 离开代码块自动关闭, 即使中间报错也会关
with open("data.json") as f:
    content = f.read()

# 大模型推理时最常见的写法: 临时关闭梯度计算, 省内存、加速
with torch.no_grad():
    output = model.generate(input_ids)
```

with 对象 as 变量:

进入代码块前「准备」, 离开代码块后自动「收尾」

常见场景①

打开文件 `open(...)`

常见场景②

`torch.no_grad()`: 推理时不需要反向传播, 关掉能提速省显存

- 不用记住原理, 先记住「看到with就是自动收尾」

class 与 self: 给数据和行为「打包」

```
class Retriever:
    def __init__(self, kb, top_k=3):
        self.kb = kb          # 把参数存到「自己」身上
        self.top_k = top_k

    def search(self, query):
        scores = [self._score(query, doc) for doc in
self.kb]
        return sorted(scores, reverse=True)[:self.top_k]

    def _score(self, query, doc):
        return len(set(query) & set(doc))

r = Retriever(kb=my_kb, top_k=5)    # 创建一个「实例」
results = r.search("李白")         # 调用实例的方法
```

class 类名:

定义一个「模板」

__init__(self, ...)

创建实例时自动执行，用来初始化数据

self

指代「这个实例自己」，访问自己的数据要写 self.xxx

r = Retriever(...)

根据模板「造出」一个具体的对象

第 1 题



```
call_model("你好", "额外1", "额外2", top_p=0.9, max_tokens=512)
```

如果函数定义是下面这样，那么 kwargs 最终会是什么？



```
def call_model(prompt, *args, temperature=0.7, **kwargs):  
    ...
```

- A** ("额外1", "额外2")
- B** {"top_p": 0.9, "max_tokens": 512}
- C** {"额外1": None, "额外2": None}
- D** ("额外1", "额外2", 0.9, 512)

第 2 题

```
with torch.no_grad():  
    output = model.generate(input_ids)
```

以下说法正确的是：

- A** with语句的作用是让代码运行得更慢，但更安全
- B** torch.no_grad()的意思是"接下来这段代码不需要计算梯度"，常用于模型推理阶段，能省显存、加速
- C** with语句只能用于打开文件，不能用于其他场景
- D** 这段代码的作用是训练模型，需要计算梯度更新参数

03

吃透 LMCC 真题代码

拿真实考题代码练手

真题 唐诗检索: retrieve_relevant_poems 全貌

```
def retrieve_relevant_poems(problem, kb, top_k=3):
    clean = re.sub(r"[.?!]", "", problem)
    q_grams = {clean[i:i+2] for i in range(len(clean)-1)}

    candidates = []
    for poem in kb:
        text = "".join(poem["paragraphs"])
        p_grams = {text[i:i+2] for i in range(len(text)-1)}
        overlap = len(q_grams & p_grams)
        candidates.append({"score": overlap, "poem": poem})

    candidates.sort(key=lambda x: x["score"], reverse=True)
    return [c["poem"] for c in candidates[:top_k]]
```

整体思路

把问题和每首诗都切成「2字词组」的集合，比较重叠数量

q_grams & p_grams

集合的「交集」运算，求两个集合共同拥有的元素

sort + lambda

按重叠数从高到低排序

最后一行

又是一个列表推导式，把排序结果的前top_k个「诗」取出来

第 3 题



```
q_grams = {clean[i:i+2] for i in range(len(clean)-1)}
```

这行代码属于哪种写法？

- ☐ A 列表推导式
- ☐ C 集合推导式

- ☐ B 字典推导式
- ☐ D 生成器表达式

第 4 题



```
overlap = len(q_grams & p_grams)
```

这里的 & 运算符，在两个集合之间表示什么操作？

- A** 并集（合并两个集合的所有元素）
- B** 交集（求两个集合共同拥有的元素）
- C** 差集（求只在其中一个集合里出现的元素）
- D** 判断两个集合是否相等

04

手搓一个迷你 RAG

把学到的写法串起来，动手写一遍

目标：写一个「不需要真实大模型」的迷你检索器

会用到的知识点： `class + self`、字典/集合推导式、`lambda`排序、默认参数——刚好是今天讲的核心内容。

思路三步： ① 把文本切成词组集合；② 遍历知识库算重叠度当作「相似度」；③ 排序取前`top_k`个。

为什么这样练： 跟刚刚真题的思路完全一样，但不需要装深度学习框架，随时能在任何环境跑起来验证。



迷你RAG: MiniRetriever

```
class MiniRetriever:
    def __init__(self, kb, gram=2, top_k=3):
        self.kb = kb
        self.gram = gram
        self.top_k = top_k

    def _grams(self, text):
        n = self.gram
        return {text[i:i+n] for i in range(len(text)-n+1)}

    def search(self, query):
        q = self._grams(query)
        scored = [
            {"score": len(q & self._grams(doc)), "doc": doc}
            for doc in self.kb
        ]
        scored.sort(key=lambda x: x["score"], reverse=True)
        return [s["doc"] for s in scored[:self.top_k]]
```

- ▶ 跟真题 一模一样的思路，只是包成了一个「类」
- ▶ `gram=2` 存到 `self.gram`，之后任何方法都能用 `self.gram` 取到
- ▶ `search` 里的 `scored` 是一次性写好的「字典列表」推导式
- ▶ 改 `gram=3` 试试结果有什么变化



05

总结

把知识串成地图

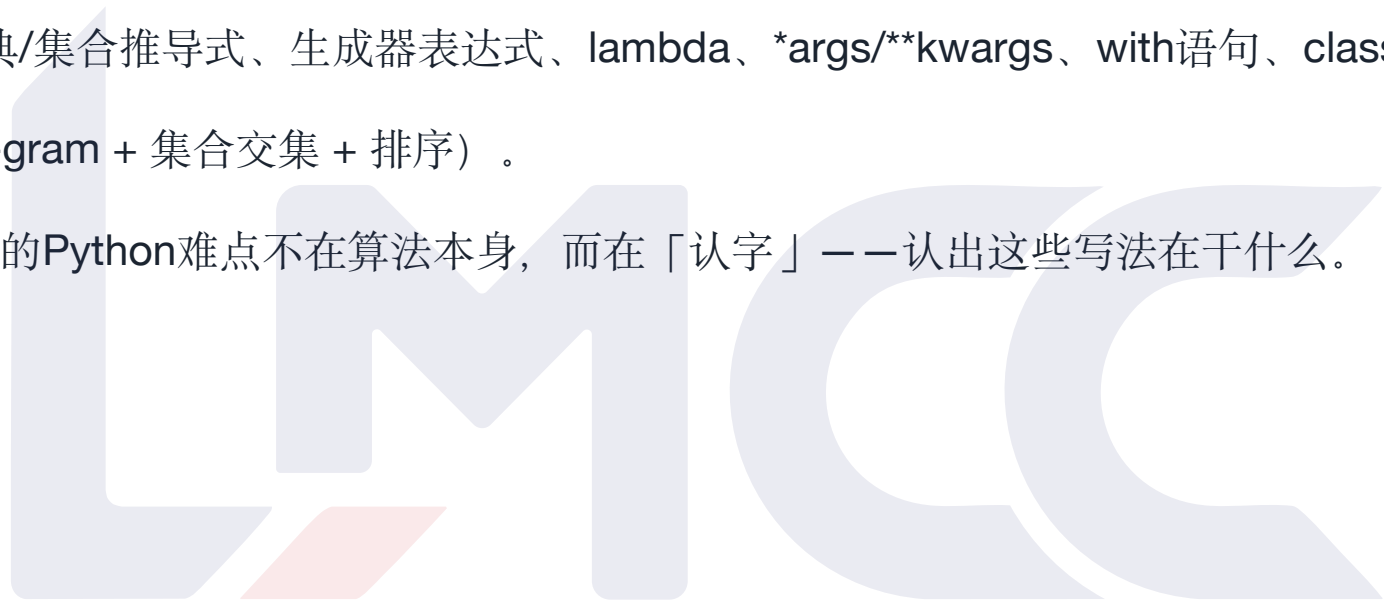
今天的知识地图

语法体检： 缩进分块、动态类型、if/for/while、def函数与默认参数、f-string。

高频写法： 列表/字典/集合推导式、生成器表达式、lambda、*args/**kwargs、with语句、class与self。

真题： 诗词检索（n-gram + 集合交集 + 排序）。

一句话记住： LMCC的Python难点不在算法本身，而在「认字」——认出这些写法在干什么。



谢谢观看

看不懂代码，就把它「翻译」成最朴素的写法——这是今天最重要的方法论